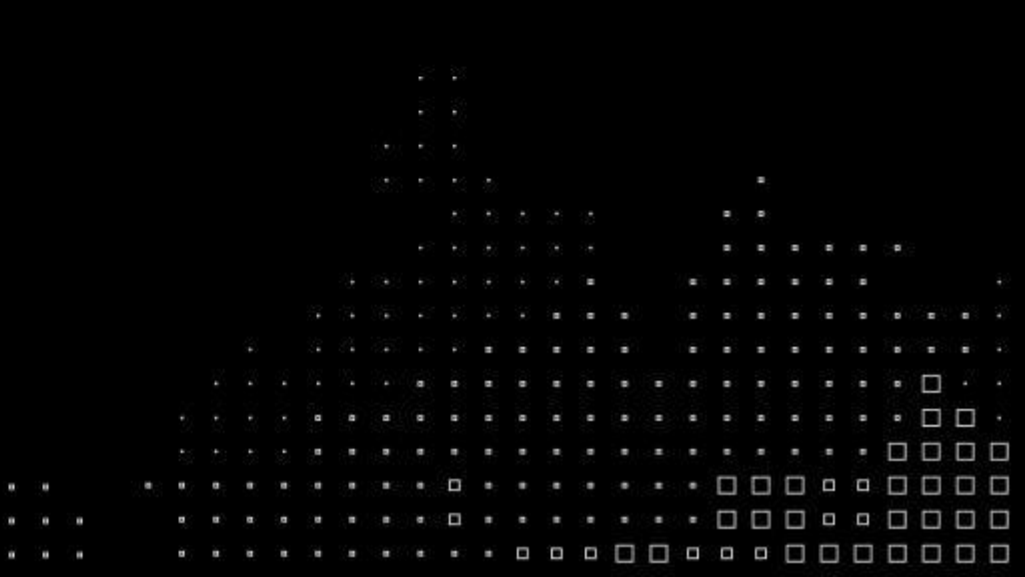


FIAP GRADUAÇÃO

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO



MODELAGEM LINEAR PARA APRENDIZADO DE MÁQUINA

Professor Me. Eng. Rodolfo Magliari de Paiva

Introdução à Linguagem de Programação Python



Antes de falarmos sobre o **Python** é importante entender que a aprendizagem de uma **linguagem de programação** deve ser precedida pela compreensão da **lógica de programação**.

A **lógica de programação** é a **organização coerente e sequencial de instruções** para que um computador **execute tarefas e resolva problemas**, funcionando como a base para criar **algoritmos**.

Um **algoritmo** é uma **sequência finita, lógica e bem definida de instruções ou regras** que, ao serem seguidas, **realizam uma tarefa, resolvem um problema ou tomam decisões automaticamente.**

Funcionam como uma "receita" para computadores, **organizando dados para realizar ações** como navegação GPS, buscas na internet ou recomendações em redes sociais.



Um algoritmo pode ser implementado em **diferentes linguagens de programação.**

Uma linguagem de programação é um **conjunto de regras e instruções** que permite ao ser humano **comunicar-se com o computador**, possibilitando a **criação de programas**, a **manipulação de dados** e a **resolução de problemas.**



Ao escrever um algoritmo em uma linguagem de programação, obtemos o **código**, que pode ser **executado pelo computador**.

A maior parte das linguagens de programação utiliza o **inglês para a escrita do código**.

Esse idioma é **essencial para programadores**, pois a maior parte do material técnico e de aprendizagem está em inglês.

Dominar o inglês **facilita a compreensão da sintaxe**, amplia as oportunidades profissionais e ajuda a acompanhar as **inovações tecnológicas**.



Os vídeos a seguir apresentam a **evolução das linguagens de programação** ao longo das décadas, impulsionada por **inovações tecnológicas**:

Evolution of Programming Languages (2001-2025):
<https://www.youtube.com/watch?v=6yYAo-iagxU>
(Acesso em 03/2026).

Most Popular Programming Languages 1955 - 2025:
https://www.youtube.com/watch?v=5yAbVkiMI_M
(Acesso em 03/2026).

E o que dizer sobre o...

Vibe Coding





FIAP

A **Vibe Coding** refere-se à prática de programar **sem compreender plenamente a lógica ou o funcionamento do código**, muitas vezes apenas copiando ou utilizando ferramentas **automatizadas e de IA**.



ChatGPT



deepseek



Copilot



Gemini



Kimi



Meta AI

Entre muitas outras!

Entre os principais **riscos**, podemos destacar:

- Falta de compreensão do código;
- Dificuldade em resolver erros;
- Dependência excessiva de ferramentas;
- Aprendizado superficial;
- Problemas em projetos mais complexos;
- Uso de soluções inadequadas ou ineficientes;
- Maior chance de erros ocultos no código...



Sobre o Python

Python é uma **Linguagem de Programação** de alto nível, orientada a objetos e de código aberto.

É muito utilizada para **desenvolvimento web**, **ciência de dados**, **machine learning** e **automação**, entre outras aplicações.



O **Python** surgiu pela primeira vez na história em **1989**, no centro de pesquisa **Centrum Wiskunde & Informatica (CWI)** (Holanda):



Criado pelo programador holandês
Guido van Rossum.

A primeira versão oficial (1.0) foi
lançada em 1994.



Guido van Rossum
1956 - Atualmente

Leitura complementar disponível em:

Python Geral

<https://docs.python.org/pt-br/3.8/faq/general.html#what-is-python>

(Acesso em 02/2025).

A partir daí, foi desenvolvido em um **esforço colaborativo** de pessoas em **vários locais do mundo**.

A sua evolução contínua depende da rede de contribuidores.



- **Vantagens:**

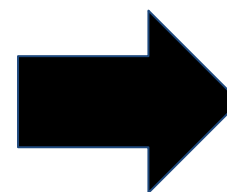
- É livre (gratuito);
- Existe uma comunidade grande de voluntários trabalhando para melhorá-lo;
- Código fácil de ler e manter;
- É compatível com uma ampla variedade de plataformas, incluindo Windows, macOS, Linux e UNIX;
- Grande quantidade de módulos e pacotes disponíveis na biblioteca padrão;
- Se adapta bem a métodos de programação orientados a objetos, procedimentais e funcionais.

- **Desvantagens:**

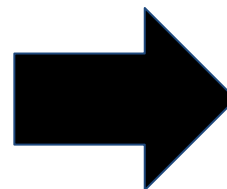
- Velocidade de execução mais lenta;
- Alto consumo de memória;
- Fraco em computação móvel;
- Propenso a erros em tempo de execução.

Para a instalação:

Primeiro devemos baixar a **Linguagem de Programação Python**:



Posteriormente, para criar e executar os algoritmos, devemos baixar o **Ambiente de Desenvolvimento Integrado** (IDE - Integrated Development Environment), **PyCharm**:



PyCharm
JETBRAINS IDE

Existem diversas IDEs para programação em Python:



ANACONDA®



Entre muitas outras!

É possível criar e executar os algoritmos em outros ambientes como:

Editor de Código, como o **VS Code**:
(é necessária uma configuração inicial).



OBS: Ao utilizar um Editor de Código ao invés de uma IDE, o desenvolvedor pode ter menos recursos integrados, como depuração facilitada, visualização de dados e suporte direto à linguagem que está utilizando. Esses fatores podem restringir algumas funcionalidades mais avançadas.

Existem diversos **Editores de Código** para **programação em Python**,
entre **outras linguagens**:



Helix

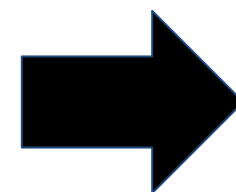


Sublime Text



Entre muitas outras!

Ambientes Interativos de Programação,
como o **Jupyter Notebook:**
(é necessária uma configuração inicial).



OBS: Ao utilizar um Ambiente Interativo de Programação, o desenvolvedor pode encontrar limitações em projetos mais complexos, como organização de código em larga escala, controle de versões e desenvolvimento de aplicações completas.

Existem diversos **Ambientes Interativos de Programação** para **programação em Python**, entre outras linguagens:

jupyterlab

Kaggle Notebooks



Apache Zeppelin

Entre muitas outras!

Compilador Online,
como o **Google Colab:**



OBS: Ao utilizar um Compilador Online ao invés de uma IDE, o desenvolvedor pode ter recursos limitados, menor desempenho no ambiente, além de depender de conexão com a internet. Esses fatores podem restringir algumas funcionalidades mais avançadas.

Existem diversos **Compiladores Online** para **programação em Python**,
entre **outras linguagens**:



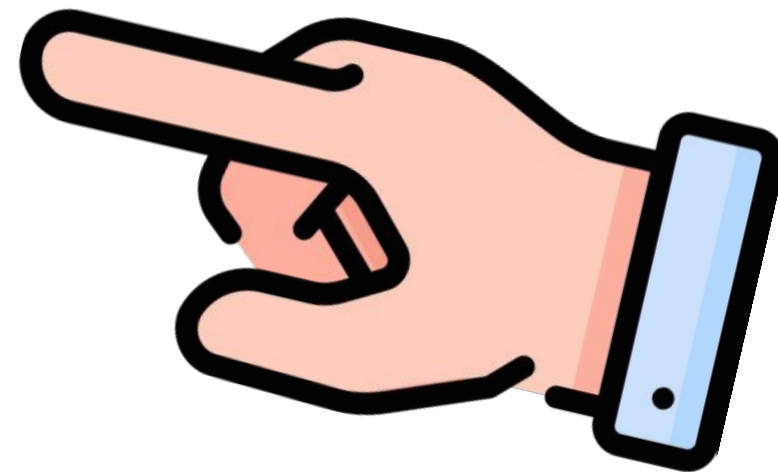
Entre muitas outras!

Por isso, em nossa disciplina, **iremos utilizar a IDE PyCharm para criar e executar os algoritmos na Linguagem de Programação Python.**

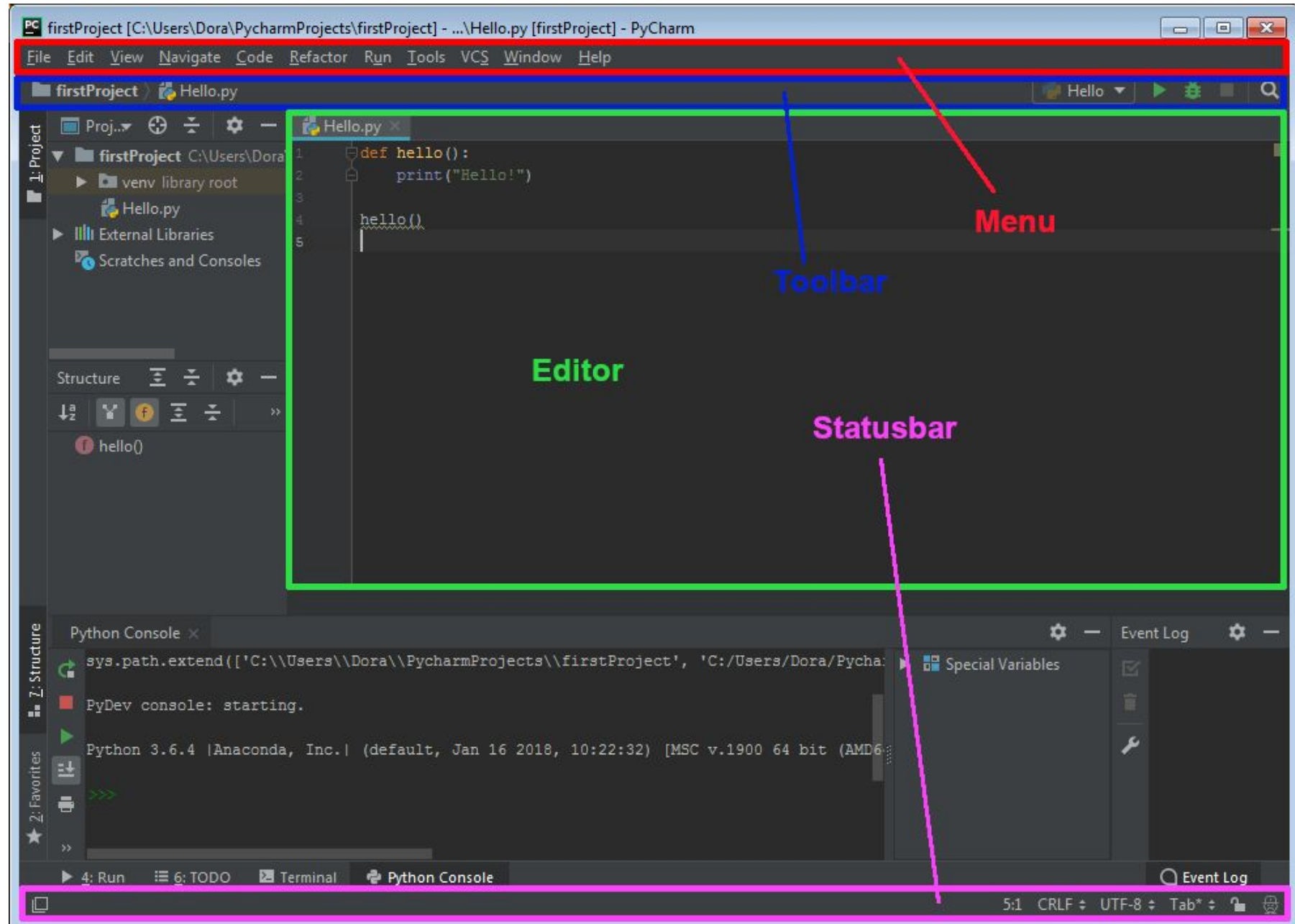
Tutorial para a instalação da Linguagem de Programação Python e da IDE PyCharm:

<https://www.youtube.com/watch?v=jYUPEIcVEgs>

(Acesso em 03/2025).



Visão Geral sobre o PyCharm



Com o PyCharm pronto para uso, podemos iniciar os nossos códigos!

Porém, é **obrigatoriamente necessário** que o **primeiro código** seja:

```
print("Hello, World!")
```



OBS: O uso de aspas simples (') ou aspas duplas (") é equivalente em termos de funcionalidade básica para strings.

Existe um mito entre programadores que diz que:

Se você não escrever "Hello, World!" no seu primeiro código, será amaldiçoado e terá bugs eternos no seu código.



HELLO WORLD!



"Olá Mundo!"

Programadores iniciantes, conhecem a maldição do "Olá Mundo"...?

Utilizando o Python como Calculadora

Para utilizar o Python como uma calculadora:

Soma:	$a + b$
Subtração:	$a - b$
Multiplicação:	$a * b$
Divisão:	a / b
Potenciação:	$a ** b$
Raíz Quadrada:	$\text{sqrt}(a)$
Raíz Enésima:	$a ** (n/m)$
Módulo:	$\text{abs}(a)$
Resto da Divisão:	$a \% b$
Quociente da Divisão:	$a // b$
Fatorial:	$\text{factorial}(a)$
Logaritmo:	$\log(a, \text{base}=b)$ ou $\log(a, b)$



OBS: Algumas funções necessitam da instalação de bibliotecas para funcionar, veremos mais adiante.

Vamos praticar!



Para escrever alguma observação ou anotação no script, sem que ele execute o comando, basta colocar no início da linha o símbolo #

Estruturas Lógicas, Condicionais e de Repetição em Python

Estruturas **lógicas, condicionais e de repetição** são conceitos fundamentais em programação que permitem **controlar o fluxo** de execução de um programa, além de garantirem que as operações sejam executadas corretamente.

Dentre suas diversas aplicações, são muito úteis para testar modelos.

Vamos entender cada um deles.

Estruturas Lógicas: As estruturas lógicas em Python são usadas para avaliar condições e tomar decisões com base em operadores lógicos.

Os principais operadores lógicos são:

- **and (e):** Verdadeiro se todas as condições forem verdadeiras.
- **or (ou):** Verdadeiro se pelo menos uma condição for verdadeira.
- **not (não):** Inverte o valor da condição (verdadeiro vira falso e vice-versa).

Estruturas Condicionais: As estruturas condicionais permitem que o programa execute diferentes blocos de código dependendo de uma condição.

Em Python, as principais são:

- **if (se):** Executa um bloco de código se a condição for verdadeira.
- **else (senão):** Executa um bloco de código se a condição do if for falsa.
- **elif (senão se):** Permite testar múltiplas condições em sequência.

Exemplo 1:

Escreva um programa em Python que verifique se uma pessoa pode dirigir com base na sua idade e na posse de carteira de motorista.

Resposta:

```
idade = 18
```

```
tem_carteira = True
```

```
if idade >= 18 and tem_carteira:
```

```
    print("Pode dirigir.")
```

```
else:
```

```
    print("Não pode dirigir.")
```

Exemplo 2:

Escreva um programa em Python que determine a situação de um aluno com base na sua nota e frequência nas aulas.

Resposta:

```
nota = 7.5
```

```
frequencia = 50
```

```
if nota >= 7 and frequencia >= 75:
```

```
    print("Aprovado!")
```

```
elif nota >= 5 and frequencia >= 75:
```

```
    print("Recuperação.")
```

```
else:
```

```
    print("Reprovado.")
```

Exemplo 3:

Crie um programa em Python que verifique o estoque de café e tome uma decisão com base na disponibilidade.

Resposta:

```
tem_cafe = False
```

```
if not tem_cafe:  
    print("Precisamos fazer mais café!")  
else:  
    print("Já temos café suficiente.")
```

Exemplo 4:

Crie um programa em Python que verifique se um número é par ou positivo.

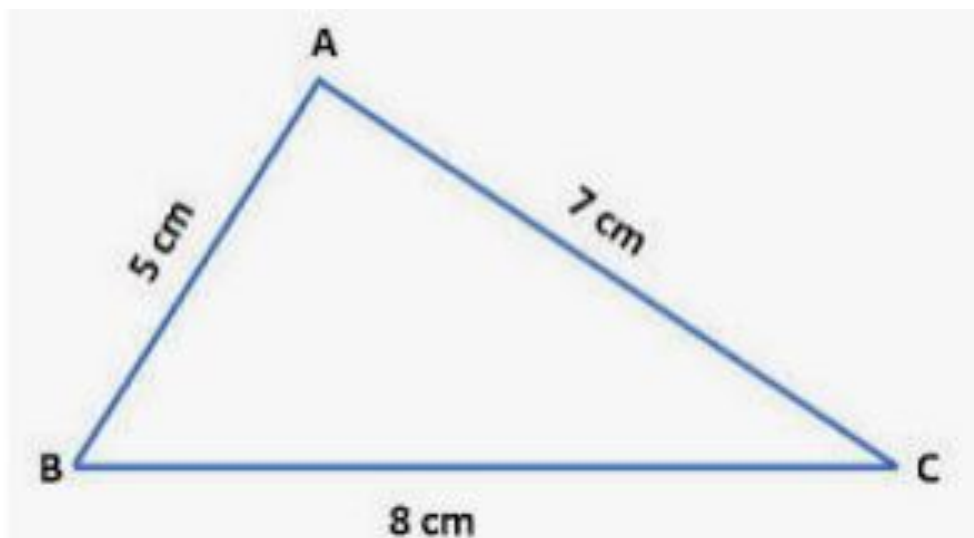
Resposta:

```
numero = int(input("Digite um número: "))  
  
if numero > 0 or numero % 2 == 0:  
    print("O número é positivo ou par")  
else:  
    print("O número não é positivo nem par")
```

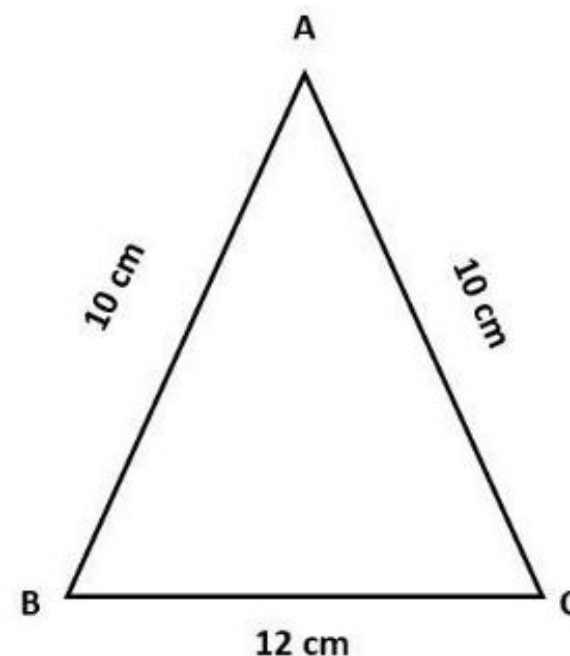
Exercício 1:

Escreva um programa em Python que verifique se o triângulo é escaleno, caso contrário, responda que não é um triângulo escaleno.

a)



b)



Resposta:

a)

a = 5

b = 7

c = 8

if a != b and b != c and a != c:

 print("É escaleno!")

else:

 print("Não é escaleno.")

b)

a = 10

b = 10

c = 12

if a != b and b != c and a != c:

 print("É escaleno!")

else:

 print("Não é escaleno.")

Exercício 2:

Escreva um programa em Python que verifique se um número x é positivo ou negativo e execute a seguinte lógica:

Se x for positivo ou zero, calcule e imprima a raiz quadrada de x .

Se x for negativo, imprima a mensagem: "O número é negativo".

a)

$$\sqrt{-16}$$

b)

$$\sqrt{81}$$

Resposta:

a)

$x = -16$

if $x \geq 0$:

`print(x**(1/2))`

else:

`print("O número é negativo")`

b)

`x = 81`

`if x >= 0:`

`print(x**(1/2))`

`else:`

`print("O número é negativo")`

Estruturas de Repetição: Também conhecidas como Laços, as estruturas de repetição permitem que um bloco de código seja executado várias vezes (loops).

Em Python, as principais são:

- **for (para):** É usado para iterar sobre uma sequência (como uma lista, string ou range) um número conhecido de vezes.
- **while (enquanto):** Repete um bloco de código enquanto uma condição for verdadeira.

Exemplo 1:

Escreva um programa em Python que imprima os números de 0 a 4, cada um precedido pela mensagem "Número:" utilizando a estrutura for.

Resposta:

```
for i in range(5):  
    print("Número:", i)
```

Exemplo 2:

Escreva um programa em Python que imprima os números de 0 a 4, cada um precedido pela mensagem "Número:", utilizando o laço de repetição `while`.

Resposta:

```
n = 0
```

```
while n < 5:
```

```
    print("Número:", n)
```

```
    n += 1
```

Exercício 1:

Escreva um programa em Python que calcule e exiba o cubo dos números inteiros de 1 a 5, utilizando a estrutura for.

Resposta:

```
for numero in range(1, 6):  
    print(numero ** 3)
```

Exercício 2:

Escreva um programa em Python que imprima os números de 1 a 100, pulando de 10 em 10, ou seja, exibindo apenas os valores 1, 11, 21, 31, ..., 91, utilizando a estrutura while.

Resposta:

```
numero = 1
```

```
while numero < 100:  
    print(numero)  
    numero += 10
```

Existem diversas outras funções e estruturas em Python para controle de fluxo, como match-case, break, continue, map, filter, e compreensões de lista, além das funções já vistas, que permitem gerenciar a execução do código de forma eficiente e adaptável a diferentes cenários de programação.

Além das Estruturas Lógicas, Condicionais e de Repetição, existem também as Estruturas de Tratamento de Exceções, que servem para gerenciamento de erros e para sinalização de problemas durante a execução do programa.

Funções no Python

Função é um bloco de código que recebe um ou mais argumentos, executa uma série de operações sobre esses argumentos e retorna um resultado.

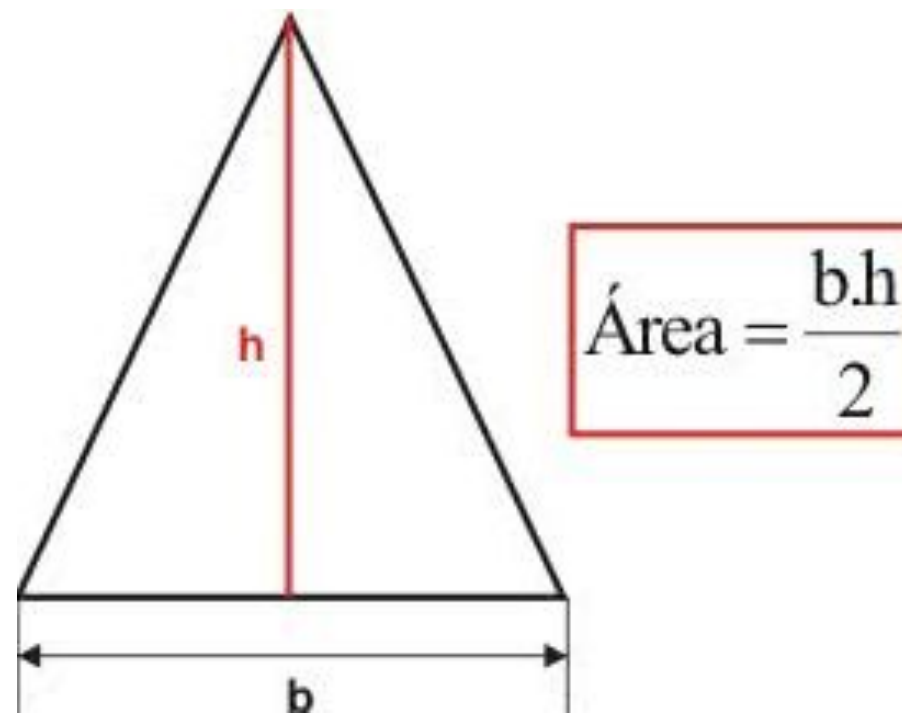
As funções são uma parte fundamental da programação em Python, e são amplamente utilizadas para **modularizar o código, reduzir a complexidade e aumentar a reutilização do código**.

OBS: Modularizar um código significa dividir um programa em partes menores, independentes e reutilizáveis, chamadas de módulos, funções ou classes, para melhorar a organização, manutenção e escalabilidade do software (capacidade de um sistema de crescer (suportar mais usuários, dados ou processamento) sem perder desempenho ou precisar ser totalmente refeito).

Exemplo 1:

Crie uma função em Python que determine a área de um triângulo para cada valor da base (b) e altura (h).

Na sequência, teste para um triângulo de base 2cm e altura 14cm.



Resposta:

```
def area_triangulo(b, h):  
    return (b * h) / 2
```

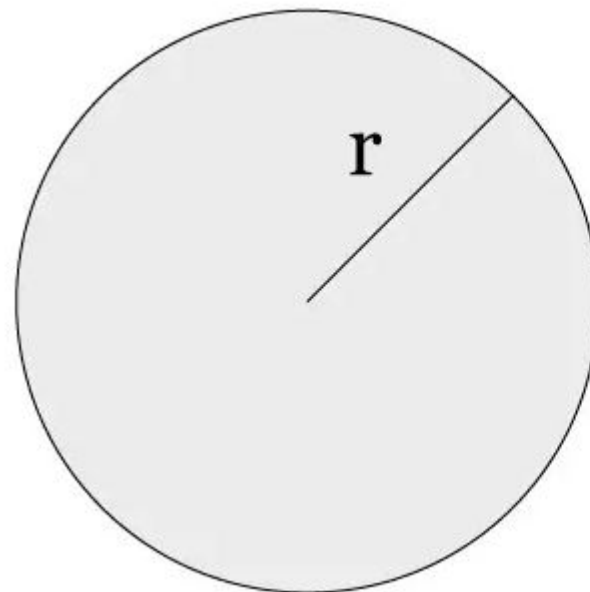
```
print(area_triangulo(2, 14))    ou    print(area_triangulo(b=2, h=14))
```

Exemplo 2:

Crie uma função em Python que determine a área de uma circunferência para cada valor de raio (r).

Na sequência, teste para uma circunferência de raio 10mm.

OBS: Utilize $\pi = 3,141593$.



$$A = \pi \cdot r^2$$

Resposta:

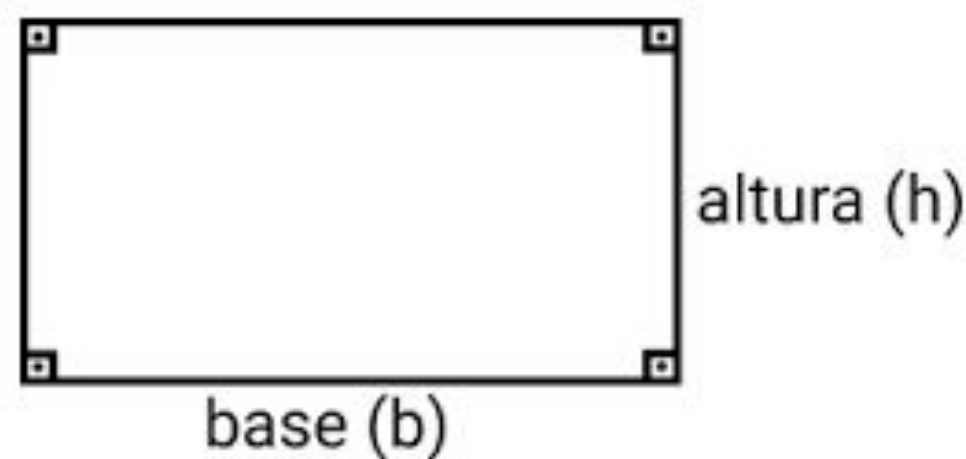
```
def area_circunferencia(r):  
    return 3.141593 * r ** 2
```

```
print(area_circunferencia(10))    ou    print(area_circunferencia(r=10))
```

Exercício 1:

Crie uma função em Python que determine o perímetro de um retângulo para cada valor de base (b) e altura (h).

Na sequência, teste para um retângulo de base 12m e altura 4m.



$$\text{Perímetro} = 2b + 2h \text{ ou } 2(b + h)$$

Resposta:

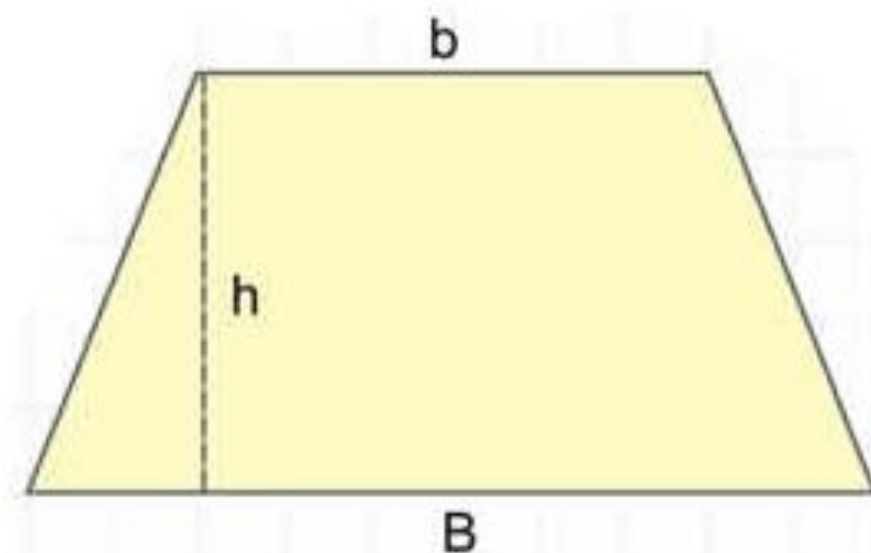
```
def perimetro_retangulo(b, h):  
    return 2 * (b + h)
```

```
print(perimetro_retangulo(12, 4)) ou print(perimetro_retangulo(b=12, h=4))
```

Exercício 2:

Crie uma função em Python que determine a área de um trapézio para cada valor de base maior (B), base menor (b) e altura (h).

Na sequência, teste para um trapézio de base maior 5m, base menor 3m e altura 4m.



$$A = \frac{(B + b) \times h}{2}$$

Resposta:

```
def area_trapezio(B, b, h):  
    return ((B + b) * h) / 2
```

```
print(area_trapezio(5, 3, 4))    ou    print(area_trapezio(B=5, b=3, h=4))
```

Objetos no Python

Um **objeto** em Python é uma **instância de classe** que **armazena dados (atributos) e comportamentos (métodos)**.

No Python, os objetos são fundamentais, pois permitem salvar, acessar e reutilizar dados de forma eficiente.

Além disso, eles podem ser manipulados e convertidos em diferentes tipos (como listas, dicionários ou DataFrames), o que oferece flexibilidade para análise e processamento de dados.

Objetos no Python são **estruturas de dados que armazenam e organizam informações** na memória do computador.

Cada tipo de objeto possui características específicas que determinam como os dados são guardados, acessados e manipulados através das funções e operações da Linguagem Python.

Os principais tipos de objetos em Python são:

- Lista;
- Tupla;
- Dicionário;
- Conjunto.



Estrutura de Dados:

Lista

'Apto'	87.1	520
--------	------	-----

índice: 0 1 2

Ordenável e Mutável

Exemplos: list(1, 2)

['Apto', 87.1, 520]

Tupla

'Apto'	87.1	520
--------	------	-----

índice: 0 1 2

Imutável

Exemplos: tuple(1, 2)

(1, 2)

1, 2

Dicionário

Não ordenável

Chave imutável

Exemplos: dict({1 : 2})

{ '2021' : 25, '2020' :

100, '2019' : 94}

chave valor

'2021'	25
'2020'	100
'2019'	94

Conjunto



Não ordenável e Mutável

Exemplos: set({1, 2})

{'Apto', 87.1, 520}

Ao criar objetos, é essencial utilizar nomes descritivos, pois isso facilita a compreensão e a manutenção do código.

Antes de criar objetos, recomenda-se:

- Nomes de objetos devem iniciar com **letras** (não números ou símbolos);
- Nomes de objetos **não devem conter espaços**;
- **Evitar** caracteres especiais (é, à, â, ç, ...);
- Letras maiúsculas e minúsculas **não são** iguais;
- Palavras-chaves (*keywords*), **não podem** ser empregadas (if, else, for, ...).

Declara-se um objeto em Python da seguinte forma: `x = 5`

OBS: No Python, a declaração de um objeto significa atribuir um valor a uma variável. A variável pode ser uma lista, um dicionário, um conjunto, uma tupla, uma função ou qualquer outro objeto que possa ser criado no PyCharm.

LISTA: É uma estrutura de dados unidimensional que pode conter valores de tipos diferentes (números, textos, outros objetos).

As listas são uma das estruturas de dados mais fundamentais em Python e são amplamente utilizadas em programação geral, análise de dados e automação.

Para criarmos um vetor fazemos:

```
lista1 = list(range(1, 8))    (cria uma lista de 1 até 7)
print(lista1)
```

```
lista2 = [2, 3, 6, 9, 8, 5, 10]
print(lista2)
```

```
lista3 = [10, "texto", 3.14, True]  
print(lista3)
```

Para localizar o elemento na posição n de uma lista, fazemos:

`NomeDaLista[n]`

Para localizar os elementos que estão a partir da posição m até a posição $p-1$ de uma lista, fazemos:

`NomeDaLista[m:p]`

OBS: Caso a posição do elemento não exista, o Python retornará o resultado `[]` (não existente).

TUPLA: É uma estrutura de dados ordenada e imutável, usada para armazenar coleções de elementos em Python.

Ela é semelhante a uma lista, mas, uma vez criada, não pode ser alterada (não é possível adicionar, remover ou modificar itens após sua criação).

Para criarmos uma tupla, fazemos:

```
tupla1 = (1, "Python", True, 3.14)  
print(tupla1)
```

```
tupla2 = 10, 20, 30  
print(tupla2)
```

DICIONÁRIO: É uma estrutura de dados flexível e associativa que armazena pares de chave-valor, onde cada valor pode ser acessado por uma chave única. Diferentemente de listas ou tuplas, que usam índices numéricos, os dicionários permitem chaves de vários tipos, enquanto os valores podem ser de qualquer tipo.

Para criarmos um dicionário, fazemos:

```
dicionario1 = {  
    "nome": "Maria",  
    "idade": 28,  
}  
print(dicionario1)
```

```
dicionario2 = {  
    "nome": "Notebook",  
    "marca": "Dell",  
    "preco": 4299.90,  
    "estoque": True  
}
```

```
print(dicionario2)
```

CONJUNTO: É uma estrutura de dados que armazena elementos únicos e não ordenados.

Ele é especialmente útil quando você precisa garantir que não haja duplicatas em uma coleção ou quando precisa realizar operações matemáticas entre conjuntos, como união, interseção e diferença.

Para criarmos um conjunto, fazemos:

```
conjunto1 = {"maçã", "banana", "laranja"}  
print(conjunto1)
```

```
conjunto2 = set([1, 2, 3, 2, 1])  
print(conjunto2)
```

Para adicionarmos um elemento, fazemos:

```
conjunto1.add("morango")
```

Para adicionarmos mais de um elemento, fazemos:

```
conjunto1.update(["morango", "abacaxi"])
```

Para removermos um elemento, fazemos:

```
conjunto1.remove("banana")
```

Para removermos mais de um elemento, fazemos:

```
conjunto1.difference_update(["abacaxi", "morango"])
```

Para realizarmos operações matemáticas, fazemos:

`conjunto3 = {1, 2, 3}`

`conjunto4 = {3, 4, 5}`

- União:

`print(conjunto3 | conjunto4)`

- Intersecção:

`print(conjunto3 & conjunto4)`

- Diferença:

`print(conjunto3 - conjunto4)`

Classificação e Localização de Objetos

Para sabermos como o Python está lendo e interpretando (classificando) um objeto, utilizamos a função:

```
print(type(NomeDoObjeto))
```

Para sabermos quais objetos criamos no Python até o momento, e que estão na memória, basta executar o comando:

```
variaveis = [var for var in dir() if not var.startswith('__')]
print(variaveis)
```

Para remover os objetos criados e que estão na memória:

```
del NomeDoObjeto
```

Utilizando o Help no Python

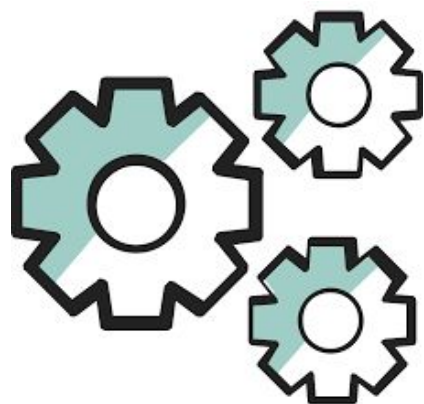
Para pedir ajuda ao Python, de modo que ele te forneça uma explicação (documentação) sobre alguma função, deve-se escrever:

```
help("NomeDaFunção")
```

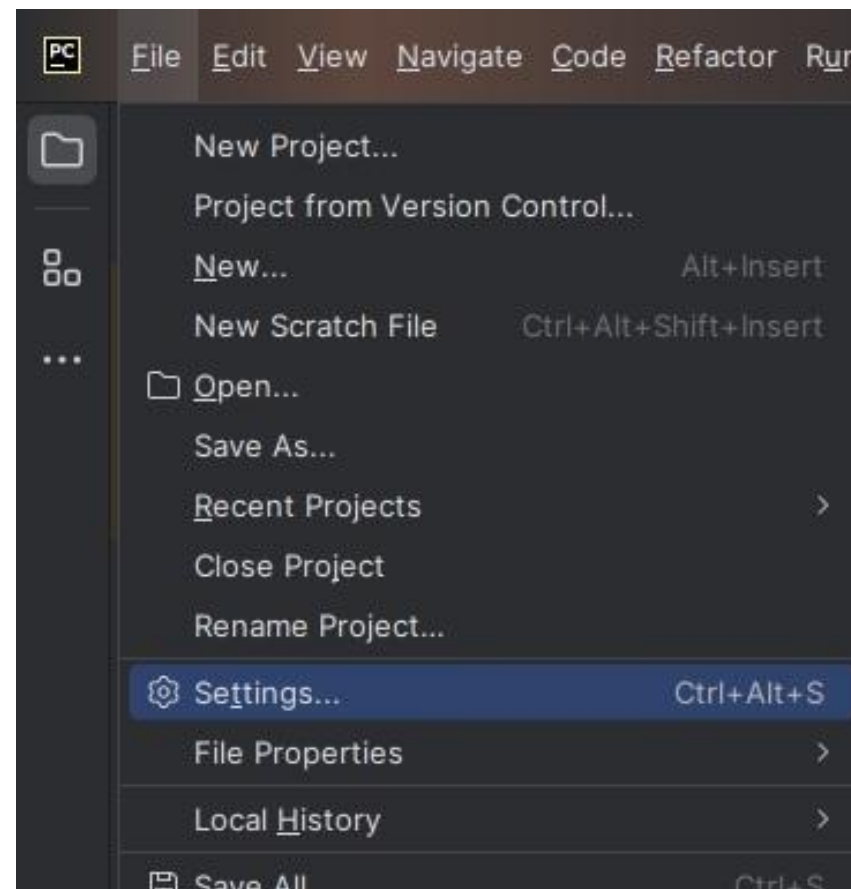


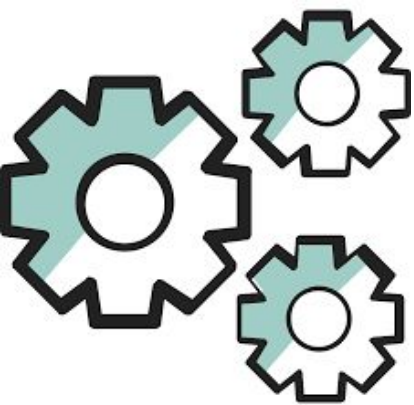
Bibliotecas no Python

Ao instalarmos o Python, automaticamente várias **bibliotecas** já são disponibilizadas para o usuário.



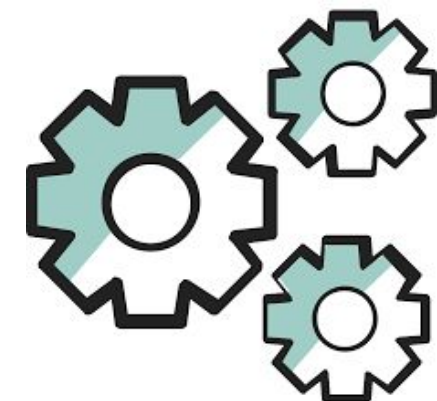
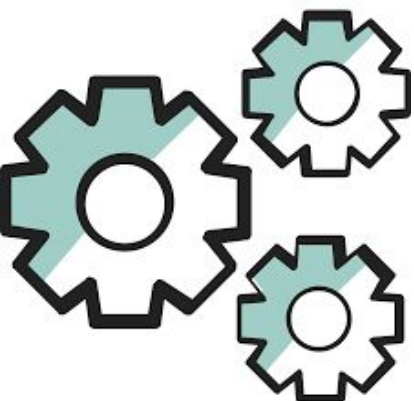
É possível verificarmos as bibliotecas existentes e seus objetivos em: File → Settings → Project: [Nome do Projeto] → Python Interpreter.

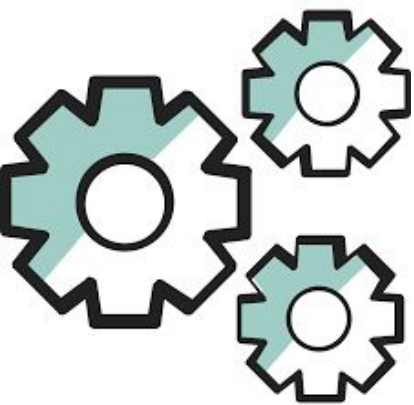




As **bibliotecas** no Python têm como objetivo fornecer um **conjunto de funcionalidades adicionais** à linguagem, desenvolvidas e mantidas pela comunidade de usuários.

Essas funcionalidades incluem desde operações básicas (como manipulação de dados com Pandas e NumPy) até visualizações sofisticadas (Matplotlib, Seaborn), técnicas de aprendizado de máquina (Scikit-Learn, TensorFlow) e integrações com APIs e ferramentas externas.





Para instalarmos alguma biblioteca via script, fazemos:

```
pip install NomeDaBiblioteca
```

OBS:

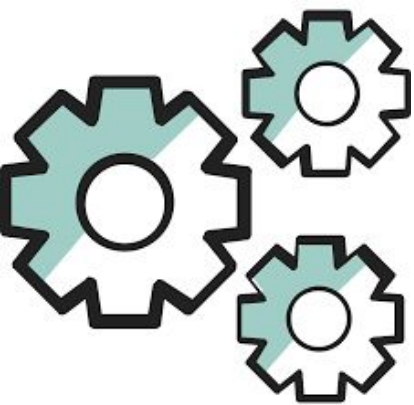
É necessário executar o comando acima no terminal integrado:

- Pressione Alt + F12 no PyCharm

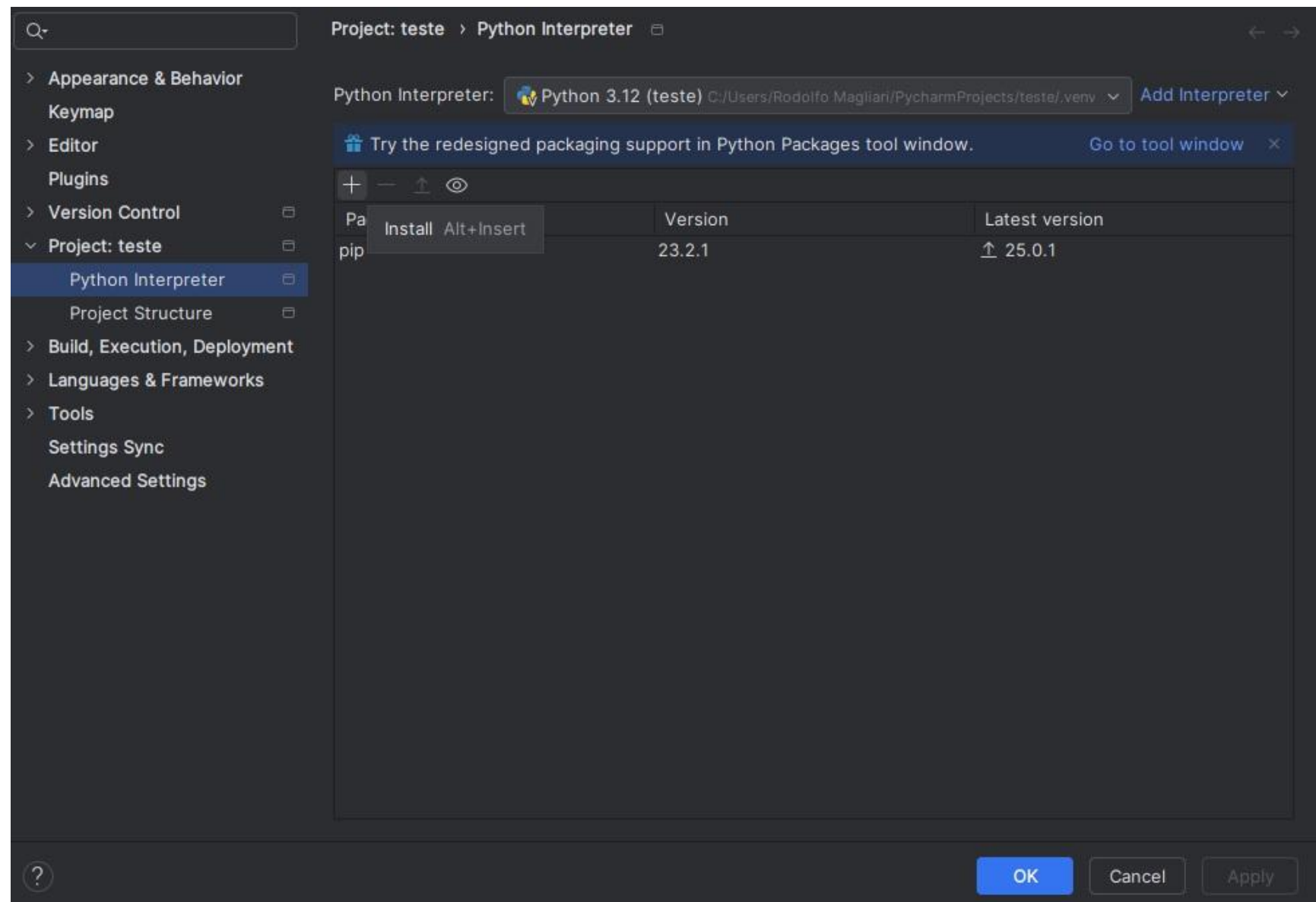
ou

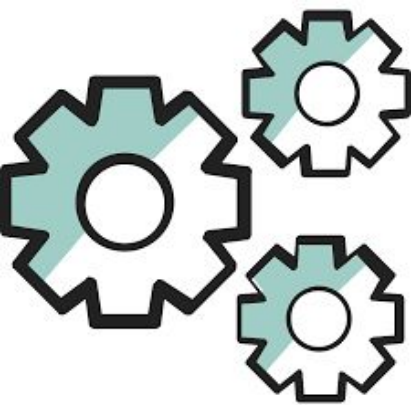
Vá até: View → Tool Windows → Terminal

- Em seguida, digite o comando e pressione Enter.



Para instalarmos
bibliotecas pela
interface gráfica
fazemos:

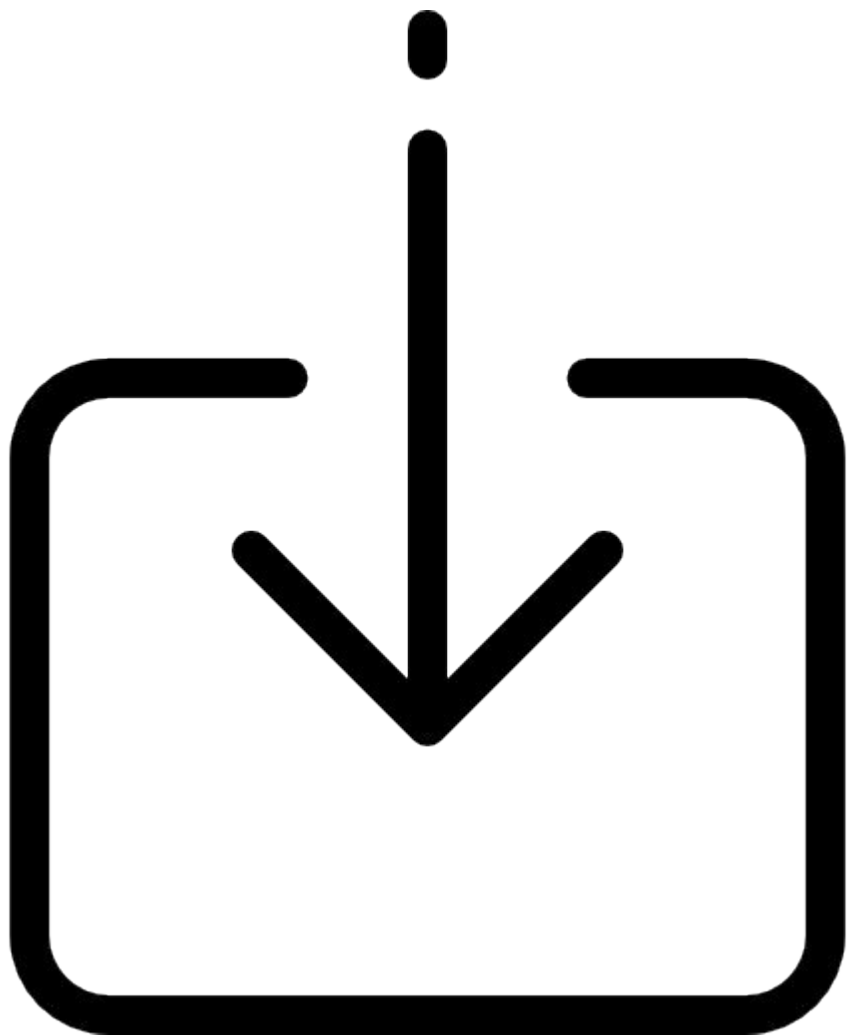




Ao instalarmos uma biblioteca, é necessário que ela seja carregada para poder funcionar, por meio de script, fazemos:

```
import NomeDaBiblioteca
```

Leitura de Dados



Para que o **Python** leia os dados que serão **importados** (para análises posteriores), são necessárias funções e/ou bibliotecas, irá depender do **formato** do arquivo:



O arquivo a ser importado deve estar no **diretório de trabalho (workspace)** do Python ou você deve indicar seu caminho completo.

Para localizar o workspace do Python em sua máquina, baixe e habilite a biblioteca os e utilize a função `os.getcwd()`:

```
import os  
print(os.getcwd())
```

Após **localizar o workspace** do Python e **colocar o arquivo no local indicado**, estamos prontos para começar a importação para a leitura de dados.

Para alterar o workspace do Python fazemos: `os.chdir("c:/novo/diretorio")`

Arquivo TXT: Formato de texto

Necessário instalar e habilitar o pacote **pandas**

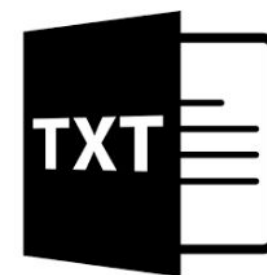
Exemplo:

```
import pandas
```

```
dados1 = pandas.read_csv("DadosAula.txt", sep=" ", header=0)  
print(dados1)
```

Caso desejemos incluir **novos dados** no arquivo original, o Python não fará a atualização automaticamente.

Nesse caso, devemos realizar a edição no arquivo original, **salvá-lo e rodar novamente o código no Python.**



Arquivo CSV: Formato .csv

Necessário instalar e habilitar o pacote **pandas**

Exemplo:

```
import pandas
```

```
dados2 = pandas.read_csv("DadosAula.csv", sep=";", header=0)  
print(dados2)
```

Caso desejemos incluir **novos dados** no arquivo original, o Python não fará a atualização automaticamente.

Nesse caso, devemos realizar a edição no arquivo original, **salvá-lo e rodar novamente o código no Python.**



Arquivo XLSX: Formato .xlsx

Necessário instalar e habilitar os pacotes **pandas** e **openpyxl**

Exemplo:

```
import pandas
```

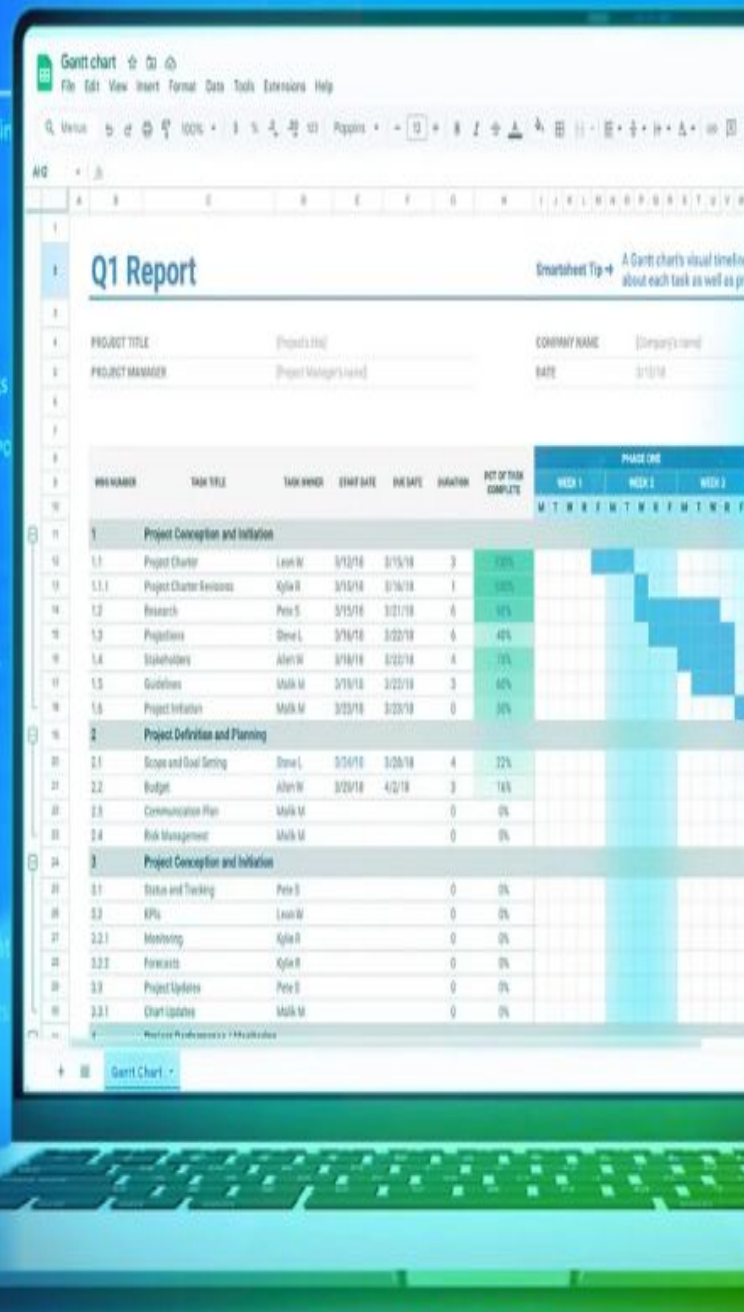
```
dados3 = pandas.read_excel("DadosAula.xlsx", engine="openpyxl")  
print(dados3)
```

Caso desejemos incluir **novos dados** no arquivo original, o Python não fará a atualização automaticamente. Nesse caso, devemos realizar a edição no arquivo original, **salvá-lo e rodar novamente o código no Python.**



Caso o arquivo Excel tenha mais do que uma aba, devemos acrescentar:

```
nome = pandas.read_excel("NomeDoObjeto.xlsx",  
sheet_name="NomeDaAba", engine="openpyxl")
```



O formato **.xlsx** é o **mais utilizado**, por ser amplamente adotado em contextos acadêmicos, científicos e corporativos.

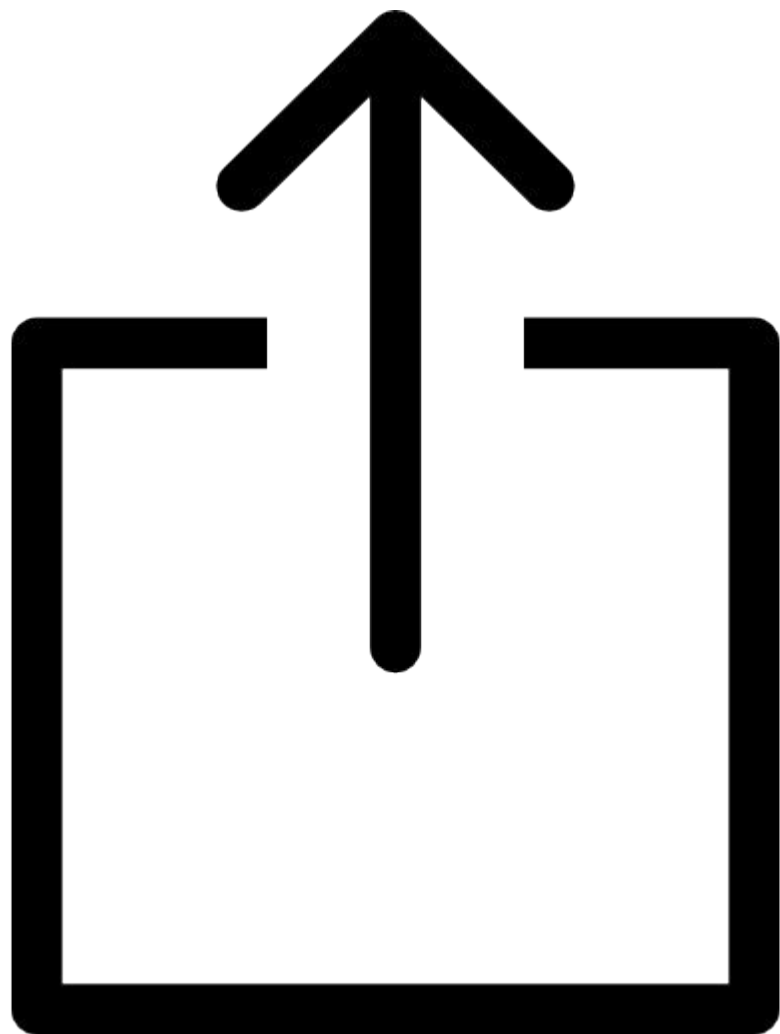
Uma vez que o **arquivo desejado já se encontra no ambiente de programação**, é possível realizar diversas operações, como **organização, manipulação, visualização e análise dos dados.**

Escrita de Dados

Escrever dados no Python significa **exportar** um arquivo criado e/ou modificado no Python para a sua máquina.



Aprender esse procedimento é fundamental, pois, em algum momento, você pode precisar enviar uma base de dados modificada no Python para alguém que não utiliza ou não tem acesso ao software, entre outras situações.



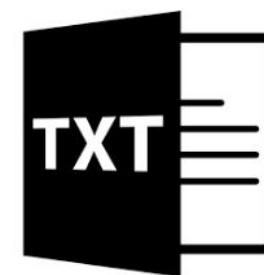
Existem várias maneiras de realizar esse processo:

- Ter um arquivo carregado no PyCharm e, posteriormente, convertê-lo para o formato desejado;
- Criar um objeto no PyCharm e, em seguida, exportá-lo para um arquivo;
- ...

Arquivo TXT: Formato de texto

Exemplo:

```
dados1.to_csv("NovoDados1.txt", sep=" ", index=False,  
header=True)
```



Arquivo CSV: Formato .csv

Exemplo:

```
dados2.to_csv("NovoDados2.csv", sep=";", index=False,  
header=True)
```



Arquivo XLSX: Formato .xlsx

Necessário instalar e habilitar o pacote **openpyxl**

Exemplo:

```
dados3.to_excel("NovoDados3.xlsx", sheet_name="DadosAula",  
index=False, engine="openpyxl")
```





Novamente, o formato **.xlsx** é o **mais utilizado**, por ser amplamente adotado em contextos acadêmicos, científicos e corporativos.

Material Complementar

Para aprofundar o tema, consulte os seguintes materiais:

https://www.ime.usp.br/~leo/mac2166/2017-1/py_introducao_funcoes.html (Acesso em 08/2025).

<https://hub.asimov.academy/tutorial/entendendo-objetos-em-python-um-guia-para-iniciantes/> (Acesso em 08/2025).

<https://repositorio.ifmg.edu.br/server/api/core/bitstreams/ba5fe2b4-a510-4a1a-9fda-7e59d758cb36/content> (Acesso em 08/2025).

<https://aws.amazon.com/pt/what-is/python/#:~:text=O%20Python%20%C3%A9%20uma%20linguagem,executada%20em%20muitas%20plataformas%20diferentes> (Acesso em 08/2025).

<https://www2.ufjf.br/deptocomputacao/files/2010/08/apresentacao.pdf> (Acesso em 08/2025).

<https://www.alura.com.br/artigos/python?srsItid=AfmBOopGQ1EvnkUjRKZiJf9gnpESTIWketYdbw47ITrxolPu4P4q7sPF> (Acesso em 08/2025).

OBRIGADO!

Contato:
profrodolfo.paiva@fiap.com.br

Copyright © **2023** Professor Me. Eng. Rodolfo Magliari de Paiva

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).